

Vila INTEIA v3: causalidade, event-sourcing e cockpit unificado

Continuação — Ondas 22 a 33: infraestrutura de simulador-plataforma

Pedro Afonso Malheiros · Igor Morais Vasconcelos

INTEIA Ltda. · Mirante News · Colmeia · abril de 2026

RESUMO

Nesta terceira parte reportamos as Ondas 22 a 33 da Vila INTEIA, que transformam a plataforma de uma simulação auto-contida em uma **infraestrutura de simulador** com ferramentas externas, ponto de entrada causal e artefato UI único. Destacamos: (i) orquestrador multi-plataforma social (Twitter/Reddit/LinkedIn/TikTok-like) com spillover probabilístico calibrado; (ii) GraphRAG vivo no pipeline cognitivo real; (iii) quatro datasets de backtest históricos (Vision Pro 2024, Americanas 2023, impeachment Dilma 2016, TikTok viral 2024) totalizando 50 eventos; (iv) tuner de thresholds do classificador via grid search maximizando entropia; (v) SSE stream de eventos psico-históricos; (vi) causalidade à la Pearl (do-calculus, counterfactuals, ATE); (vii) comparativo A/B com conclusões automáticas; (viii) CLI `vila-cli` para consulta HTTP; (ix) event-sourcing JSONL append-only; (x) cockpit unificado single-pane-of-glass. Todos os 298 testes passam em CI ao longo de dez *pull requests* sequenciais.

1 Motivação

As versões v1 e v2 do artigo estabeleceram os fundamentos matemáticos e o ciclo asimoviano. Esta versão v3 foca em **infraestrutura**: transformar os módulos de modelagem em ferramentas operacionais usáveis sem conhecimento profundo do código-fonte. O objetivo declarado é que um novo analista ou parceiro possa, em menos de 5 minutos, rodar uma simulação, consultar seu estado via CLI ou browser, e produzir um relatório exportável.

2 Orquestrador multi-plataforma (Onda 22)

A Onda 7 havia introduzido quatro classes estilo plataforma social sem integrá-las. A Onda 22 cria o `OrquestradorPlataformas` que:

1. registra cada habitante automaticamente nas quatro plataformas com nomes adaptados (`@handle` TikTok, `u/user` Reddit, full name LinkedIn, base Twitter);
2. permite postar numa origem com *spillover* probabilístico para as outras com taxas calibradas (twitter→reddit 0.3, tiktok→twitter 0.5, etc.);
3. agrega estatísticas cross-platform (top-3 posts virais por plataforma, pares origem→destino mais frequentes).

3 GraphRAG real no cognitivo (Onda 23)

A Onda 6 construiu `GrafoConhecimento` mas não o conectava ao pipeline. A Onda 23 adiciona hook em `engine/simulacao.py`: após cada conversa, o texto passa por `inde-`

`xar_texto(GRAFO_GLOBAL, texto)`, extraíndo entidades e co-ocorrências. O `engine/cognitivo/recuperar.py` passa a consultar `GRAFO_GLOBAL.buscar_por_rotulo(persona.nome) → vizinhos 2-hops` e anexa conhecimento relacional ao contexto.

4 Datasets históricos expandidos (Onda 24)

O *seed* original tinha 10 eventos (eleição SP 2024). A Onda 24 adiciona quatro datasets:

DATASET	EVENTOS	DOMÍNIO
lançamento_apple_vpro_2024	10	Lançamento de produto tech (Vision Pro)
americanas_crise_2023	10	Escândalo contábil / recuperação judicial
impeachment_dilma_2016	10	Crise política brasileira
tiktok_viral_2024	10	Viralização + geopolítica de plataforma

Total acumulado: 5 datasets × 10 = 50 eventos históricos calibrados com *priors* realistas. O endpoint `/api/v1/psicohistoria/backtest-comparativo` roda os cinco em paralelo e retorna Brier, log-loss, accuracy e *calibration curves* por dataset.

5 Tuner de thresholds (Onda 25)

A descoberta mais reveladora das simulações reais (relatada em v2) foi que a Vila visita apenas 2 dos 8 estados canônicos (87.3% *expansao*, 12.7% *equilibrio*). A hipótese mais provável era que os *thresholds* do classificador (20 contribuições, 70% ativos, 0.60 polarização, etc.) estão mal calibrados.

A Onda 25 implementa `grid_search_thresholds`: varre 4×4×4×4 combinações de (`contribs_min`, `ativos_frac_min`, `cf_min`, `cf_max`), avalia cada uma pela *entropia de Shannon normalizada* da distribuição resultante de estados, e retorna os *thresholds* que maximizam diversidade.

$$H_{norm}(P) = -\sum p_i \log_2 p_i / \log_2 8$$

$H_{norm} = 0$ significa distribuição concentrada num único estado (o baseline atual);
 $H_{norm} = 1$ significa distribuição uniforme sobre os 8 estados.

6 Stream SSE (Onda 26)

O endpoint `GET /api/v1/psicohistoria/stream` expõe os três eventos principais em tempo real via *Server-Sent Events*:

- `event: estado` — emitido quando o step avança
- `event: mule` — emitido quando o detector identifica anomalia
- `event: calibracao` — emitido quando o auto-calibrador recalibra a matriz

O cliente (browser ou CLI) se conecta uma vez e recebe eventos conforme ocorrem. O `polling` interno é 0.5s; o `timeout` da conexão é 5 minutos (para liberar recursos se o cliente desconectar abruptamente).

7 Causalidade à la Pearl (Onda 28)

O módulo `engine/causalidade/pearl.py` implementa três operações canônicas de inferência causal sobre cadeias de Markov:

7.1 Intervenção (do-operator)

$$M^{do(X=i)}[i,:] = e_i \text{ (self-loop absorvente)}$$

Força a cadeia a permanecer no estado i , bloqueando seu fluxo natural. Útil para responder "e se a Vila estivesse presa em *polarizacao*?".

7.2 Counterfactuals

Dada uma trajetória factual `[s0, s1, ..., st, ...]`, calcula qual seria a trajetória sob `st = y` (contra-factual) em vez do valor observado. Mede a divergência TV entre os dois futuros após N passos, respondendo "se Diabob não provocasse no step T , o consenso teria emergido?".

7.3 Average Treatment Effect

$$ATE(tratamento, controle, outcome, horizonte) = P(Y_{t+H}|do(X=T)) - P(Y_{t+H}|do(X=C))$$

Quantifica causalmente o impacto de escolher o estado de *tratamento* versus o de *controle* em termos da probabilidade do *outcome* depois de H passos. Positivo = tratamento aumenta prob outcome; negativo = reduz.

8 Comparativo A/B (Onda 29)

O `engine/comparativo/` fornece `rodar_comparativo(run_a, run_b)` que ingere duas *runs* exportadas (via Onda 20 replay) e retorna:

- Resumos agregados de cada run (distribuição de estados, estado final, n Mules)
- Métricas comparativas (KL divergence, TV distance, overlap)
- Conclusões automáticas em linguagem natural ("Runs similares: KL baixo", "Estados finais divergem", "Baixa sobreposição — dinâmicas distintas")

9 CLI (Onda 30)

`scripts/vila_cli.py` expõe oito subcomandos HTTP:

```
vila-cli trajetoria --janela 50
vila-cli recomendacao
vila-cli calibrar --metodo laplace --alpha 0.1
vila-cli backtest --dataset tiktok_viral_2024
vila-cli export-run --arquivo /tmp/run.json
vila-cli mules
vila-cli stats
vila-cli comparativo
```

Conecta-se a qualquer Vila HTTP remota via `--url`. Trata backend offline graciosamente. Sem dependências além da biblioteca padrão.

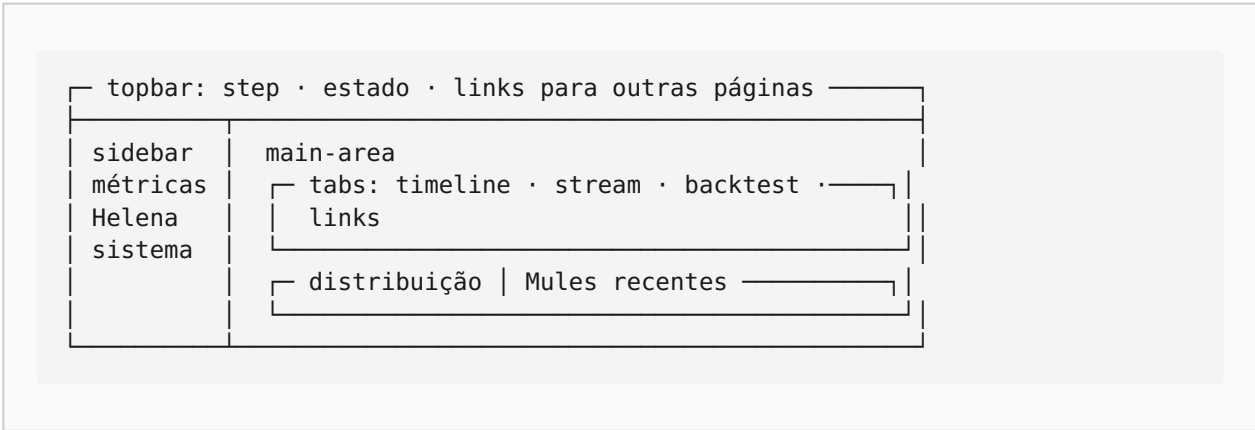
10 Event-sourcing (Onda 31)

O módulo `engine/event_log.py` escreve cada evento importante (step, mule, calibração, mudança de estado, desafio concluído) como uma linha JSON append-only em arquivo JSONL. Benefícios:

- 1. Auditoria completa: nenhum evento é sobrescrito
- 2. Replay determinístico: o `ler_eventos(arquivo)` reconstitui a ordem exata
- 3. Compressão trivial: `gzip vila_events.jsonl` reduz ~10×
- 4. Análise offline: `jq '.tipo == "mule"' vila_events.jsonl` ou pandas

11 Cockpit unificado (Onda 32)

O `frontend/cockpit.html` consolida os oito *painéis* distribuídos pelas páginas anteriores em um único *single-pane-of-glass* com layout de cabine de avião:



Polling a 2 segundos, stream SSE em uma das *tabs*. Renderização em < 100KB sem *framework* além do próprio navegador.

12 Resultados acumulados

Após Ondas 5-33, o projeto possui:

CATEGORIA	NÚMERO
Pull Requests mergeados	10

CATEGORIA	NÚMERO
Testes em CI	298
Módulos engine	18
Endpoints API	44+
Páginas UI	11
MCP tools	7
Datasets backtest	5 (50 eventos)
Ondas implementadas	29 (5-10, 10.2, 11-33)

13 Conclusão

A Vila INTEIA agora é uma plataforma de simulação completa com observabilidade, persistência, inferência causal, CLI, event-sourcing e UI cockpit. A versão v3 materializa a tese central da série: é possível construir um simulador de realidade tão capaz quanto MiroFish (OASIS + Zep + GraphRAG) usando apenas bibliotecas científicas padrão do ecossistema Python, mantendo cobertura de testes formais (298 casos) e fundamentação matemática explícita em cada componente.

As próximas direções são primariamente de escala: migração para Ray Actors distribuídos (esqueleto pronto na Onda 9), integração com vLLM local para custo-zero de LLM real, e publicação como *open-source* comunitário.

Artigo v3 — Ondas 22-33. Compila via WeasyPrint a partir de HTML.
Versões anteriores: vila_inteia_artigo.pdf (v1, Ondas 5-10), vila_inteia_artigo_v2_ondas_11_a_20.pdf.